DESARROLLO WEB · UD 01

Introducción a PHP

Itinerario de prácticas

Realiza las siete actividades en orden. En cada carpeta encontrarás un archivo `ENUNCIADO.md` y los archivos HTML y PHP de partida que tendrás que completar.

Preparación del entorno

1. Instala XAMPP e inicia Apache. Estas prácticas no requieren iniciar MariaDB.
2. Copia las siete carpetas numeradas en `htdocs/UD1/`.
3. Abre el archivo HTML indicado en cada práctica desde `http://localhost/UD1/NOMBRE_CARPETA/ARCHIVO`. En la práctica 02 abre `EJ1.php` mediante `localhost`.
4. Lee primero el enunciado, desarrolla el código pendiente y comprueba el resultado en el navegador. No abras los archivos PHP con doble clic.

Recorrido

1. Mi primer formulario con GET
2. Ticket de compra
3. Datos personales y checkbox
4. IMC y validaciones
5. Matrícula y horarios
6. Concesionario y descuentos
7. Generador y corrector de tests

PRÁCTICA 01

01 · Mi primer formulario con GET

Objetivo

Crear una página HTML que envíe un nombre y una edad a un programa PHP mediante **GET**, y mostrar un saludo generado en el servidor.

Actividad

1. Completa `miPrimeritoFormulario.html`: añade un campo `edad` numérico (entre 0 y 130) y un botón de envío. Mantén el campo `nombre` y la acción hacia `miPrimeritoProcesar.php`.
2. Completa `miPrimeritoProcesar.php`: recupera `nombre` y `edad` desde `$_GET`, valida que el nombre no esté vacío y que la edad sea un entero dentro del intervalo indicado.
3. Muestra un mensaje con el formato «Hola, Ana. Tienes 19 años.» o informa al usuario si alguno de los datos no es válido.
4. Evita mostrar directamente etiquetas HTML introducidas como nombre: trata los datos antes de incluirlos en el HTML de respuesta.

Prueba

Accede a `http://localhost/UD1/01_Primer_formulario_GET/miPrimeritoFormulario.html`, envía valores válidos y después prueba con la edad fuera de rango y con el nombre vacío. Observa también los parámetros que aparecen en la URL.

Pistas del temario: formularios GET, `$_GET`, condicionales y `htmlspecialchars()`.

PRÁCTICA 02

02 · Ticket de compra: arrays y cálculos

Objetivo

Generar con PHP un ticket de caja a partir de un **mapa asociativo** de productos/precios y un **array indexado** con las cantidades compradas.

Actividad

1. Trabaja sobre `EJ1.php`. Puedes utilizar los cinco productos y las cantidades suministrados o modificar los datos para probar tu programa. Ambos arrays han de corresponderse por el orden de los productos.
2. Recorre el mapa de productos con `foreach`. Recupera en cada iteración la cantidad de la posición correspondiente y calcula `cantidad × precio`.
3. Genera filas HTML con cantidad, producto, precio unitario y subtotal. No muestres filas de productos con cantidad 0.
4. Acumula el **total de la compra** y añádelo al final de la tabla.
5. Desglosa la parte correspondiente al **IVA del 21 % ya incluido** en los precios: no debes sumar el IVA otra vez al total final.

Comprobación

Con las cantidades `[2, 2, 1, 0, 0]`, deben aparecer **tres productos**. Comprueba el total manualmente y vuelve a ejecutar tras cambiar una cantidad.

Pistas del temario: arrays asociativos e indexados, `foreach`, acumuladores, operadores aritméticos y `round()`.

PRÁCTICA 03

03 · Formulario de datos personales

Objetivo

Completar un formulario HTML con controles de distintos tipos y procesar sus datos desde PHP mediante **POST**.

Actividad

Completa `radioCheckbox.html` con todos estos campos, manteniendo los nombres indicados para que PHP pueda recuperarlos:

Campo / `name`	Control y opciones
`nombre`	Texto de máximo 20 caracteres.
`apellidos`	Texto de máximo 20 caracteres.
`edad`	`select` con «Menos de 20 años», «Entre 20 y 39 años», «Entre 40 y 59 años», «60 años o más»; utiliza esos mismos textos como valores.
`peso`	Número en kg con decimales, de 20 a 500.
`sexo`	Radio con valores `masculino` y `femenino`.
`estado-civil`	Radio con valores `soltero`, `casado` y `otro`.
`aficiones[]`	Checkbox con valores `cine`, `literatura`, `tebeos`, `deporte`, `musica`, `television`.

Incluye botones **Enviar** y **Borrar**. El formulario debe enviar por `POST` a `porcesarDatos.php` (conserva el nombre de archivo indicado).

Completa `porcesarDatos.php` para leer los campos, comprobar que los obligatorios y las opciones seleccionadas son válidos, mostrar nombre y apellidos en un `<h1>`, edad, peso, sexo y estado civil en párrafos, y las aficiones seleccionadas en una lista `<ul>`. Si no se marca ninguna afición, muestra un mensaje en lugar de generar errores.

Prueba

Envía el formulario con una afición, con varias y sin ninguna. Comprueba que al pulsar **Borrar** se limpian los controles y que el resultado no muestra código HTML introducido en los campos de texto.

Pistas del temario: `$_POST`, `select`, radio, checkbox con `[]`, `isset()`, `foreach` y `htmlspecialchars()`.

PRÁCTICA 04

04 · Calculadora de IMC con validación

Objetivo

Procesar datos numéricos recibidos por un formulario y realizar cálculos **solo después de validar** la petición y los campos.

Actividad

1. Completa `IMC.html` con `nombre` (1–20 caracteres), `edad` (entero entre 0 y 130), `altura` (entero entre 50 y 300 **cm**) y `peso` (decimal entre 20 y 500 **kg**). Añade un botón de envío y conserva el método `POST` y la acción hacia `porcesarIMC.php`.
2. En `porcesarIMC.php` verifica el método HTTP, la presencia de los cuatro campos y los tipos/rangos indicados. Si algún dato no es válido, muestra un mensaje y no realices cálculos.
3. Convierte la altura a metros, calcula el **IMC = peso / (altura en metros)²** y muéstralo con dos decimales.
4. Calcula también una **estimación didáctica** de pulsaciones máximas con `220 - edad`. No debe presentarse como una medición clínica.
5. Muestra un resultado que incluya el nombre del usuario de forma segura.

Prueba

Prueba con valores válidos y también con altura 0, peso fuera de rango, ausencia de nombre y acceso directo al archivo PHP sin enviar el formulario.

Pistas del temario: `$_SERVER['REQUEST_METHOD']`, `$_POST`, `filter_var()`, condicionales y `round()`.

PRÁCTICA 05

05 · Matrícula y horario semanal

Objetivo

Recoger varias asignaturas mediante casillas de verificación, consultar sus horarios y calcular la carga horaria semanal utilizando arrays multidimensionales.

Actividad

1. En `matricula.html`, completa las casillas con `name="asignaturas[]"` y los valores `Ingles`, `DAW`, `DIW`, `DWEC`, `DWES`, `IPE_II`, `Proyecto` y `Optativa`. Envía por `POST` a `procesarMatricula.php`.
2. Estudia `horario.php`, que contiene los días de la semana, los tramos de media hora y el mapa de asignaturas con sus intervalos. **No hace falta inventar un horario nuevo**.
3. En `procesarMatricula.php`, recupera el array recibido y comprueba que se ha seleccionado al menos una asignatura y que todos sus nombres aparecen en los datos del horario.
4. Muestra para cada asignatura elegida sus días y horas. Algunas asignaturas tienen **dos intervalos en un mismo día**: ambos deben mostrarse y sumarse.
5. Calcula la suma de horas semanales de todas las asignaturas matriculadas. Evita contabilizar una misma asignatura más de una vez.

Ampliación

Dibuja una tabla con los días como columnas y los tramos horarios como filas. Rellena únicamente las celdas correspondientes a asignaturas matriculadas y aplica un color distinto a cada asignatura. Puedes usar las matrices `$diasSemana` y `$horasHorario` ya suministradas.

Prueba

Prueba con una asignatura, con varias, con `DWES` (tiene dos tramos el martes) y sin marcar ninguna casilla.

Pistas del temario: `$_POST`, arrays con `[]`, `foreach` anidados, `strtotime()` o conversión de horas a minutos, condicionales y tablas HTML.

PRÁCTICA 06

06 · Configurador de coches y descuentos

Objetivo

Desarrollar un configurador que valide las opciones de un vehículo, calcule su precio y desglose el descuento y el IVA.

Datos de partida

`componentes.php` ya contiene el mapa con las **opciones y precios sin IVA** de modelos (Compacto, Sedán, SUV), motores (Gasolina 1.6L, Diesel 2.0L, Eléctrico 75 kWh), colores (Blanco, Negro, Azul Metálico, Rojo), llantas (16" Aleación, 18" Aleación, 20" Deportivo), equipamientos (Básico, Confort, Premium), ocho accesorios opcionales y los códigos de descuento. Utiliza esos datos sin cambiarlos.

Actividad

1. Completa `formularioConcesionario.html` con cinco grupos de **radio obligatorios**: `Modelo`, `Motor`, `Color`, `Llantas` y `Equipamiento`. Cada valor HTML debe coincidir con una clave del mapa correspondiente de `componentes.php`.
2. Añade los **ocho accesorios** como checkbox con `name="Accesorios[]"`. Añade la cantidad de vehículos (`name="cantidad"`, entero de 1 a 5) y el código opcional (`name="codigo_descuento"`). Envía por POST a `procesarConcesionario.php`.
3. En PHP, comprueba que las cinco selecciones obligatorias son válidas; admite que no se elija ningún accesorio y evita contabilizar duplicados. Valida la cantidad y no aceptes opciones que no aparezcan en el mapa inicial.
4. Suma el precio de todos los componentes y accesorios para calcular el **precio por vehículo sin IVA**. Multiplica por el número de vehículos.
5. Si se ha introducido un código válido, aplica el porcentaje indicado en el mapa. Si el código no existe, informa al usuario y continúa sin descuento.
6. Sobre la **base resultante después del descuento**, calcula el IVA (21 %) y el total con IVA. Muestra un desglose claro: componentes, accesorios, unidades, precio unitario, base inicial, descuento, base tras descuento, IVA y total.

Prueba

Prueba una compra sin accesorios ni descuento; otra con varios accesorios y código válido; otra con código incorrecto y otra enviando un formulario con un grupo obligatorio ausente.

Pistas del temario: `require_once`, arrays asociativos, checkbox, validaciones, porcentajes, `foreach` y salida HTML segura.

PRÁCTICA 07

07 · Generador y corrector de tests

Objetivo

Crear una aplicación que seleccione preguntas aleatorias de un repositorio y corrija un test para obtener una nota entre 0 y 10.

Archivos de trabajo

- `generadorDeTests.html`: formulario de configuración del test.
- `repositorioPreguntas.php`: **banco de partida** con tres asignaturas y diez preguntas de cada una, con sus opciones. Debes completar el mapa `$repositorioRespuestas` con las respuestas correctas, manteniendo las mismas claves de asignatura y pregunta.
- `generadorDeTests.php`: genera el test de la asignatura seleccionada.
- `comprobarTest.php`: corrige el test enviado.

Actividad

1. Completa el formulario para elegir **Matemáticas, Historia o Ciencias** y un número entero de preguntas entre **1 y 5**, enviando por POST a `generadorDeTests.php`.
2. Comprueba que la asignatura y el número de preguntas recibidos son válidos. Selecciona aleatoriamente el número solicitado de preguntas **sin repetir ninguna**.
3. Genera una página HTML con las preguntas y sus posibles respuestas como botones radio. Añade un botón **Corregir test** que envíe por POST a `comprobarTest.php`.
4. Transmite también la asignatura y los identificadores de las preguntas seleccionadas para que el corrector sepa qué respuestas comparar. **No envíes al navegador el mapa de respuestas correctas.**
5. En el corrector, valida que los datos pertenecen al repositorio, cuenta **aciertos y fallos** (una pregunta en blanco cuenta como fallo) y muestra una **nota de 0 a 10** calculada como `10 × aciertos / total de preguntas`.
6. Cuida que los textos de pregunta y opción se inserten con seguridad en el HTML y que no se admitan preguntas ni respuestas inventadas por el navegador.

Prueba

Genera tests de 1 y 5 preguntas; comprueba que cambian las preguntas al repetir la operación; deja una sin responder y revisa la nota. Las respuestas correctas están en el mapa que debes completar, no en el formulario HTML.

Pistas del temario: arrays multidimensionales, `array_rand()`, `foreach`, `$_POST`, campos ocultos, comprobación de claves y validación de opciones.